MINI PROJECT REPORT •

Sales Performance Dashboard Using Power BI

Project Overview & Specifications

Objective: Develop an interactive Power BI dashboard to analyze sales performance, customer details, product categories, and monthly revenue using data from Excel or a Kaggle dataset.

Project Parameter	Details & Specifications
Tools Required	Power BI Desktop, Microsoft Excel / CSV, Kaggle Sales Dataset (Superstore Sales Dataset: `Superstore_Dirty_Data.csv`)
Project Tasks	1. Import data into Power BI Desktop. 2. Clean and transform data using Power Query Editor. 3. Build a data model and create relationships. 4. Create measures and calculated columns using DAX. 5. Design interactive dashboards using standard and custom visualizations. 6. Apply filters, slicers, and drill-through features. 7. Publish the report to Power BI Service. 8. Configure data refresh and share the dashboard.
Expected Outcome	Interactive Sales Dashboard, KPI Cards (Sales, Profit, Orders), Monthly Sales Trend, Category-wise Sales Analysis, Region-wise Sales Map, Top 10 Products/Customers, Dynamic filtering using slicers.
Deliverables	Power BI (`.pbix`) file, Dataset (Excel/CSV), Project Report, Presentation.

Task 1: Import Data into Power BI Desktop

Step 1.1: Select the Text/CSV Data Connector

Open **Power BI Desktop**. On the **Home** ribbon tab, click on the **Get data** dropdown menu to expand the list of common data sources, and select **Text/CSV** to import data from a text or CSV file.

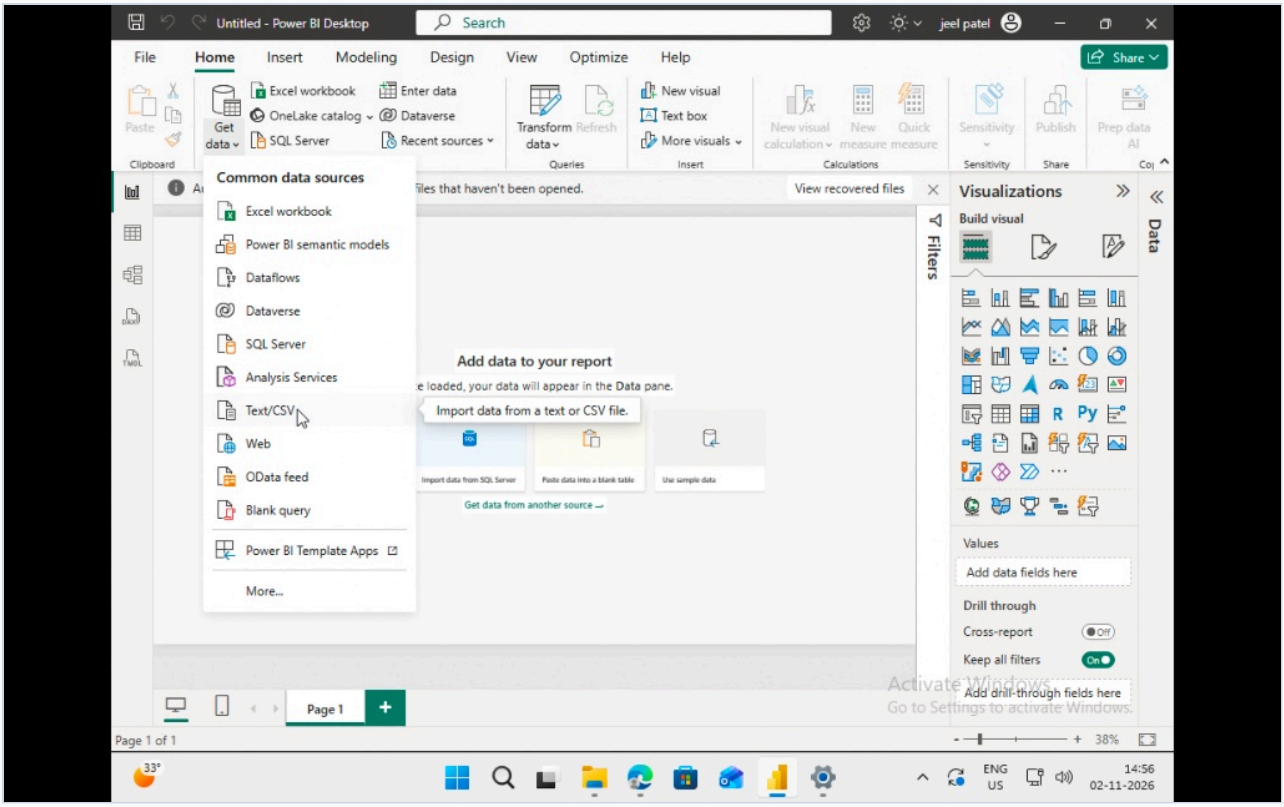


Figure 1.1: Selecting Get Data > Text/CSV from the Home ribbon in Power BI Desktop (14:56).

Step 1.2: Locate and Open the Superstore Dataset

In the file explorer dialog box, navigate to the local **Downloads** folder, select the dataset file named `Superstore_Dirty_Data.csv`, and click **Open**.

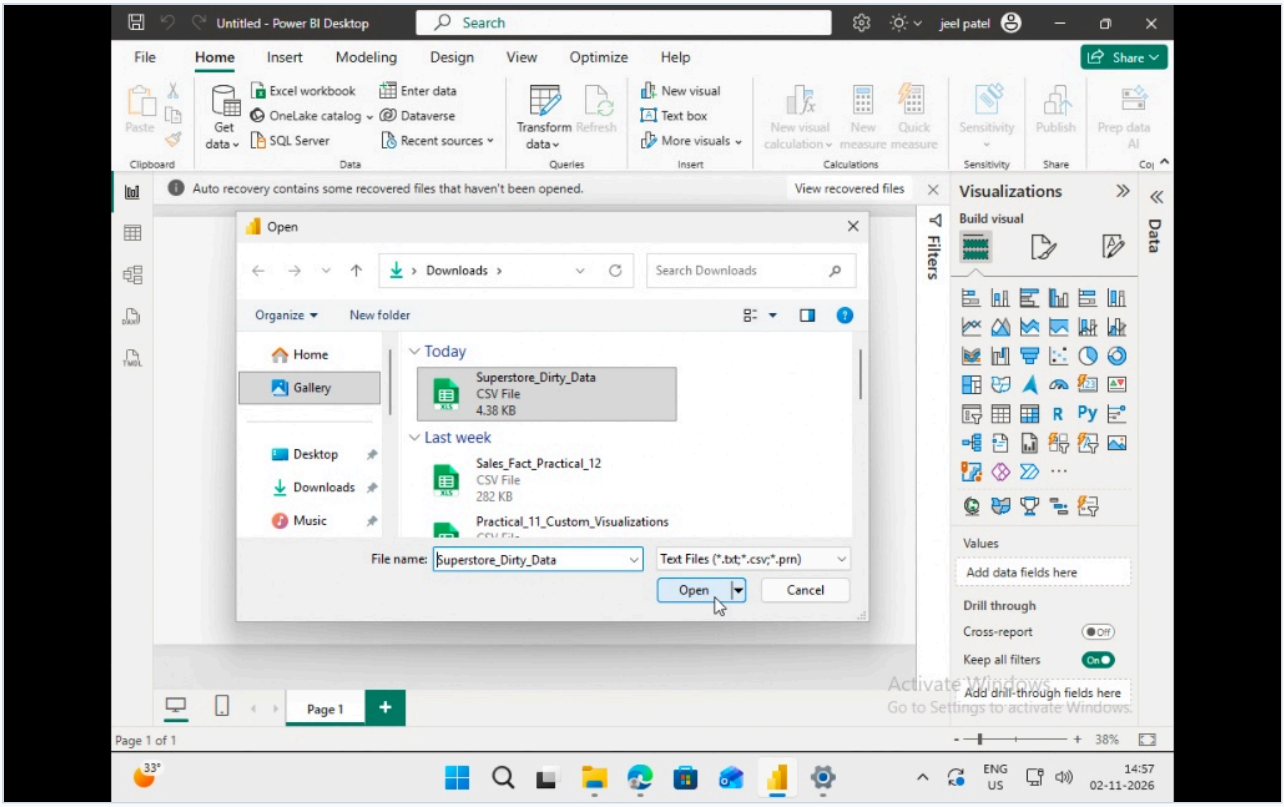


Figure 1.2: Selecting Superstore_Dirty_Data.csv in the file dialog window (14:57).

Step 1.3: Preview the Dataset and Launch Power Query Editor

Power BI displays a preview window of `Superstore_Dirty_Data.csv` with File Origin set to `1252: Western European (Windows)` and Delimiter set to `Comma`. Instead of loading the raw dirty dataset directly, click **Transform Data** to open the Power Query Editor for data cleaning.

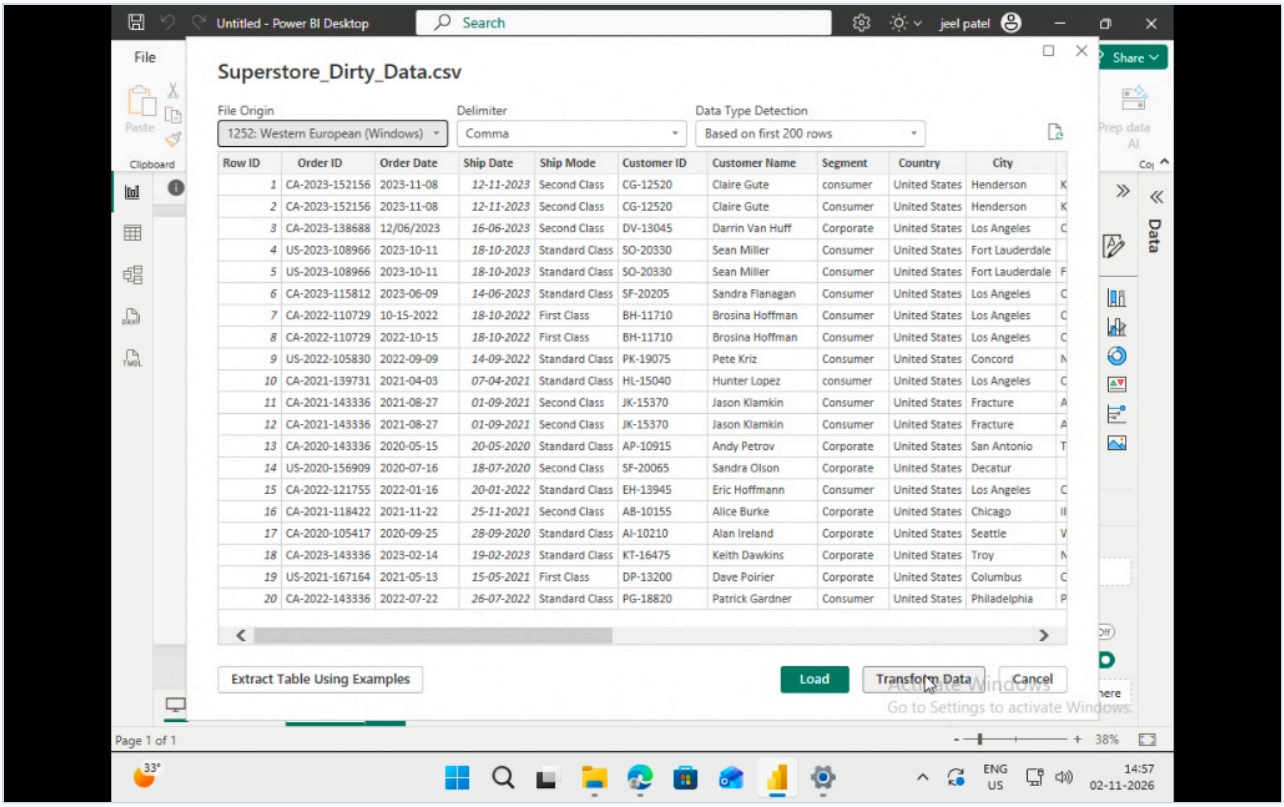


Figure 1.3: Previewing the raw CSV records and clicking Transform Data (14:57).

Task 2: Clean and Transform Data Using Power Query Editor

Step 2.1: Clean Currency Text in the Sales Column

Inside the **Power Query Editor**, select the **Sales** column, which contains inconsistent text strings such as `261.96 USD`. Open **Transform > Replace Values**, enter `261.96 USD` in **Value To Find** (or remove the text suffix), and click **OK** so the column can be converted into a clean numeric type.

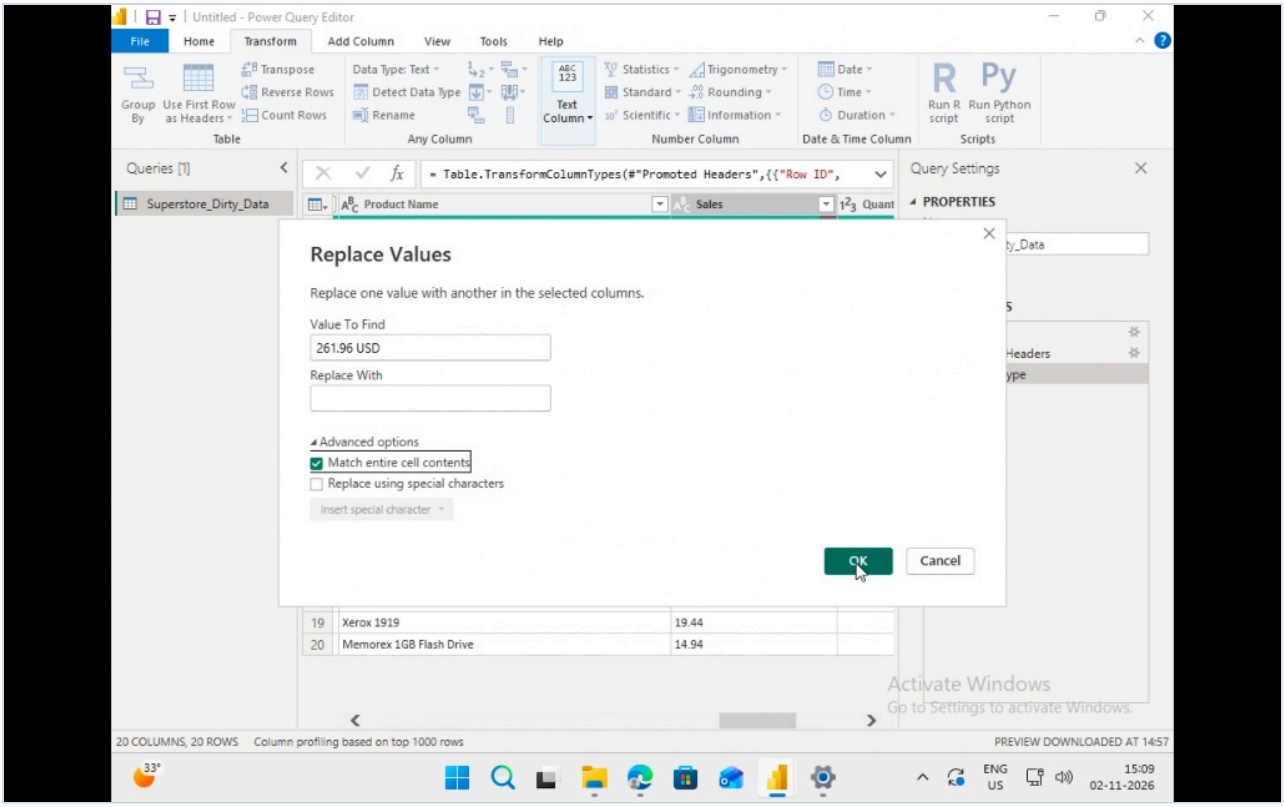


Figure 2.1: Using Replace Values on the Sales column in Power Query Editor (15:09).

Step 2.2: Standardize Date Data Types

Select the date columns (`Order Date` and `Ship Date`). On the **Transform** tab, click the **Data Type** dropdown and select **Date** to standardize all date records into a uniform date format.

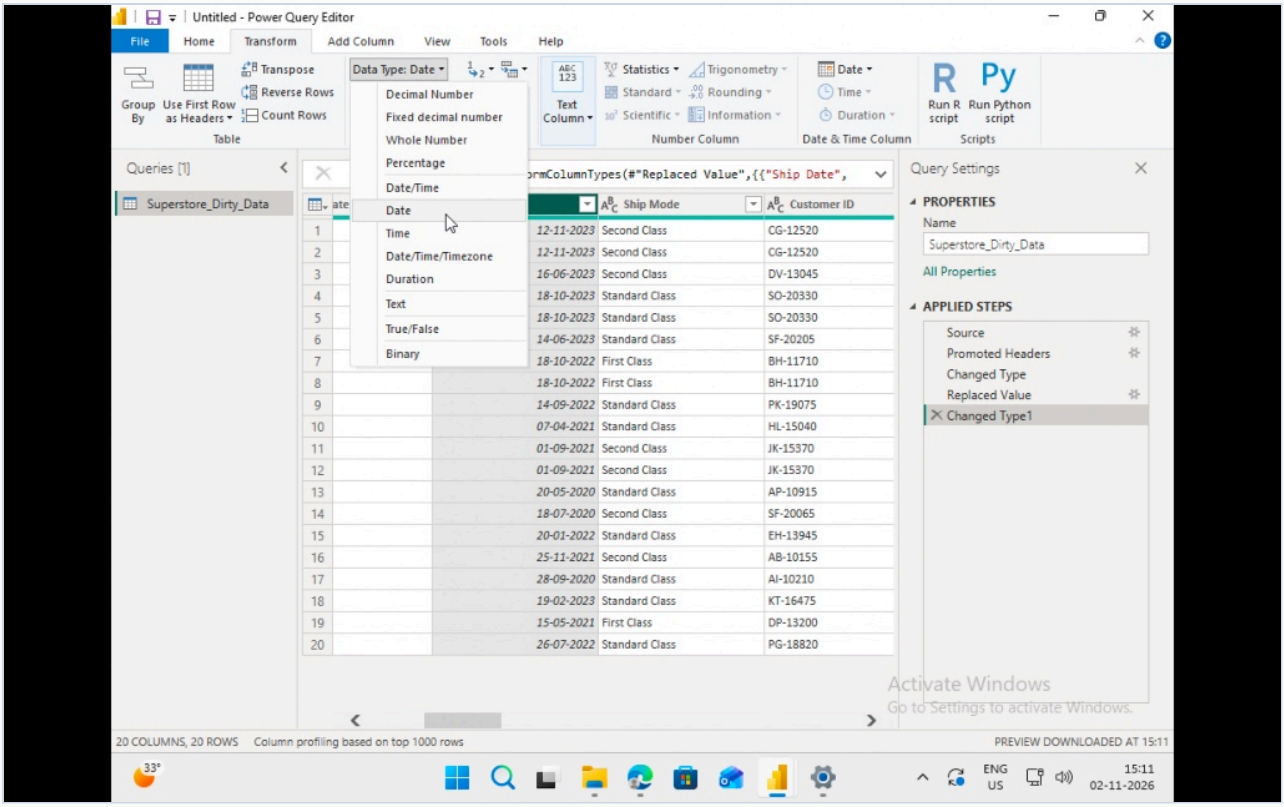


Figure 2.2: Setting the column Data Type to Date in the Transform ribbon (15:11).

Step 2.3: Trim, Clean, and Format Inconsistent Text

To fix extra whitespace and inconsistent capitalization in text columns (such as `Segment`, `Country`, and `State`), navigate to **Transform > Format** and apply **Trim**, **Clean**, and **Capitalize Each Word**.

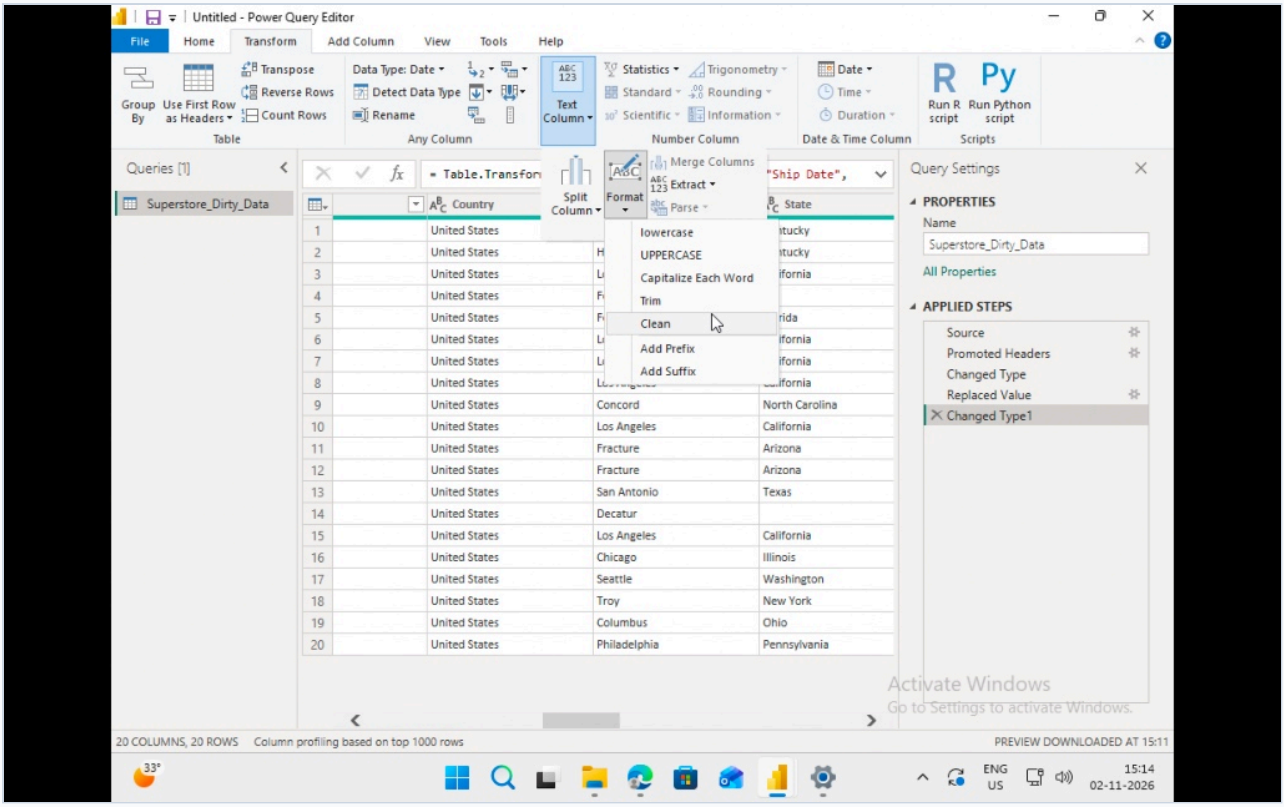


Figure 2.3: Using Transform > Format > Clean and Trim to standardize text columns (15:14).

Step 2.4: Replace Blank/Missing Values in the Sales Column

Select the **Sales** column to handle missing values. Open **Transform > Replace Values**, leave **Value To Find** blank (to target empty cells), enter `0` in **Replace With**, and click **OK**.

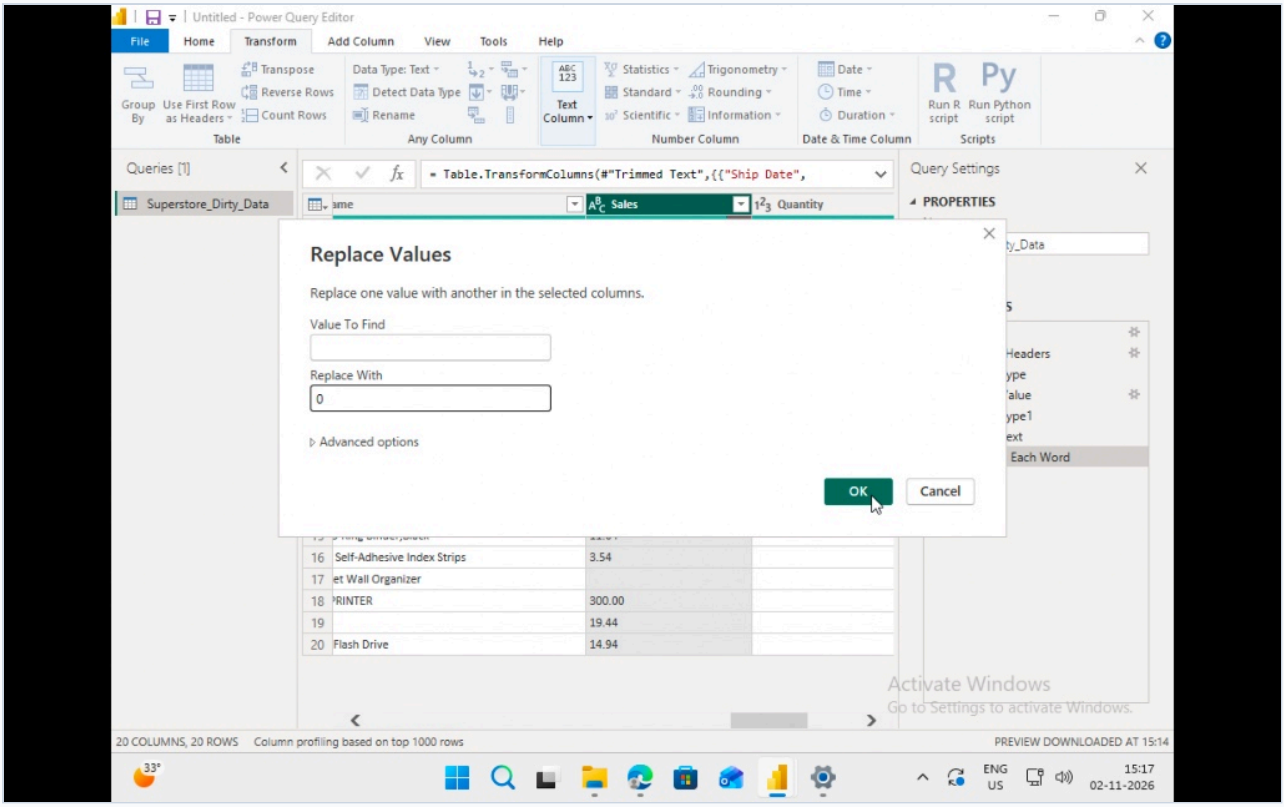


Figure 2.4: Replacing blank values in the Sales column with 0 (15:17).

Step 2.5: Replace Null Values in the Profit Column

Select the **Profit** column (set to `Decimal Number`) where missing entries appear as `null`. Open **Replace Values**, enter `null` in **Value To Find**, enter `0` in **Replace With**, and click **OK**.

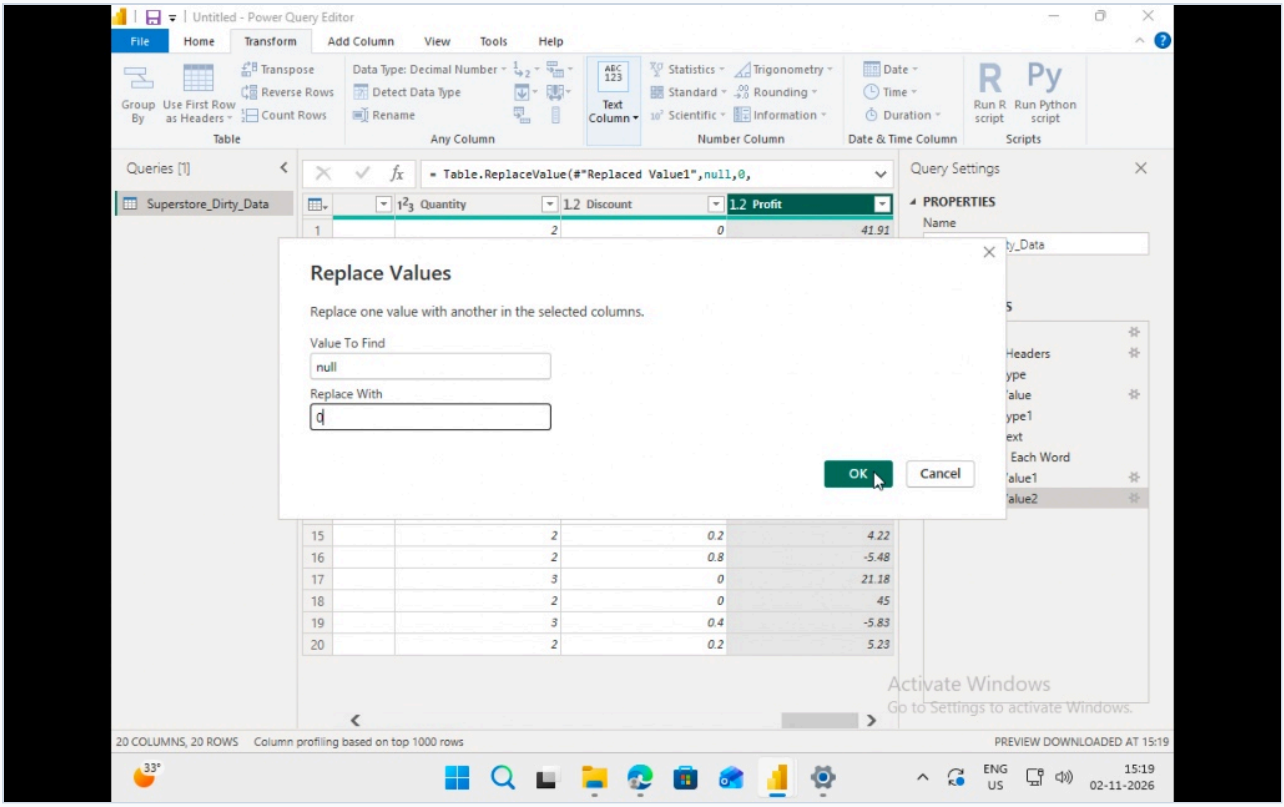


Figure 2.5: Replacing null entries in the Profit column with 0 (15:19).

Step 2.6: Handle Missing Entries in the State Column

Select the **State** column to resolve missing geographical state names. Open **Replace Values**, leave **Value To Find** empty, enter `unkown` in **Replace With**, and click **OK**.

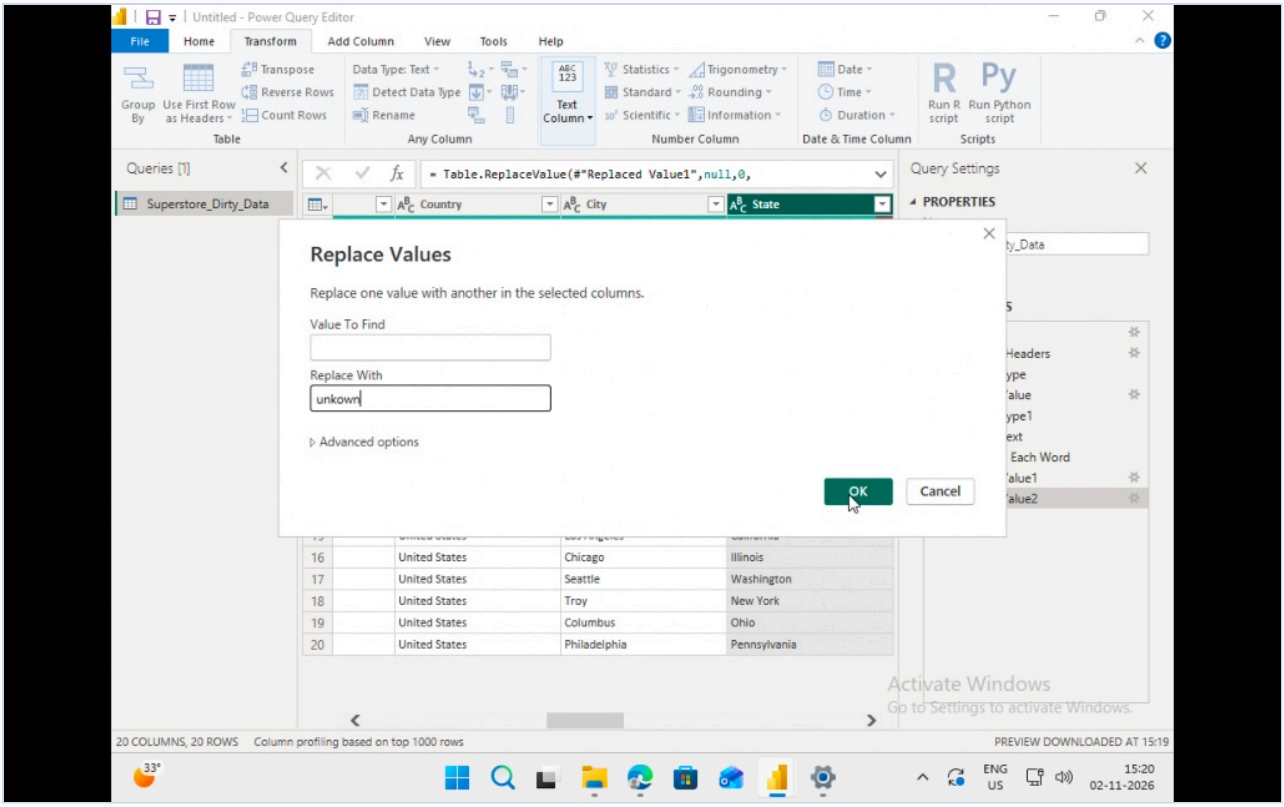


Figure 2.6: Filling missing State values using Replace Values (15:20).

Step 2.7: Create a Merged Custom Column for Full Location

Switch to the **Add Column** tab and click **Custom Column**. Name the new column `Full Location` and enter the Power Query concatenation formula: `= [City] & ", " & [State] & ", " & [Country]`. Confirm that no syntax errors are detected and click **OK**.

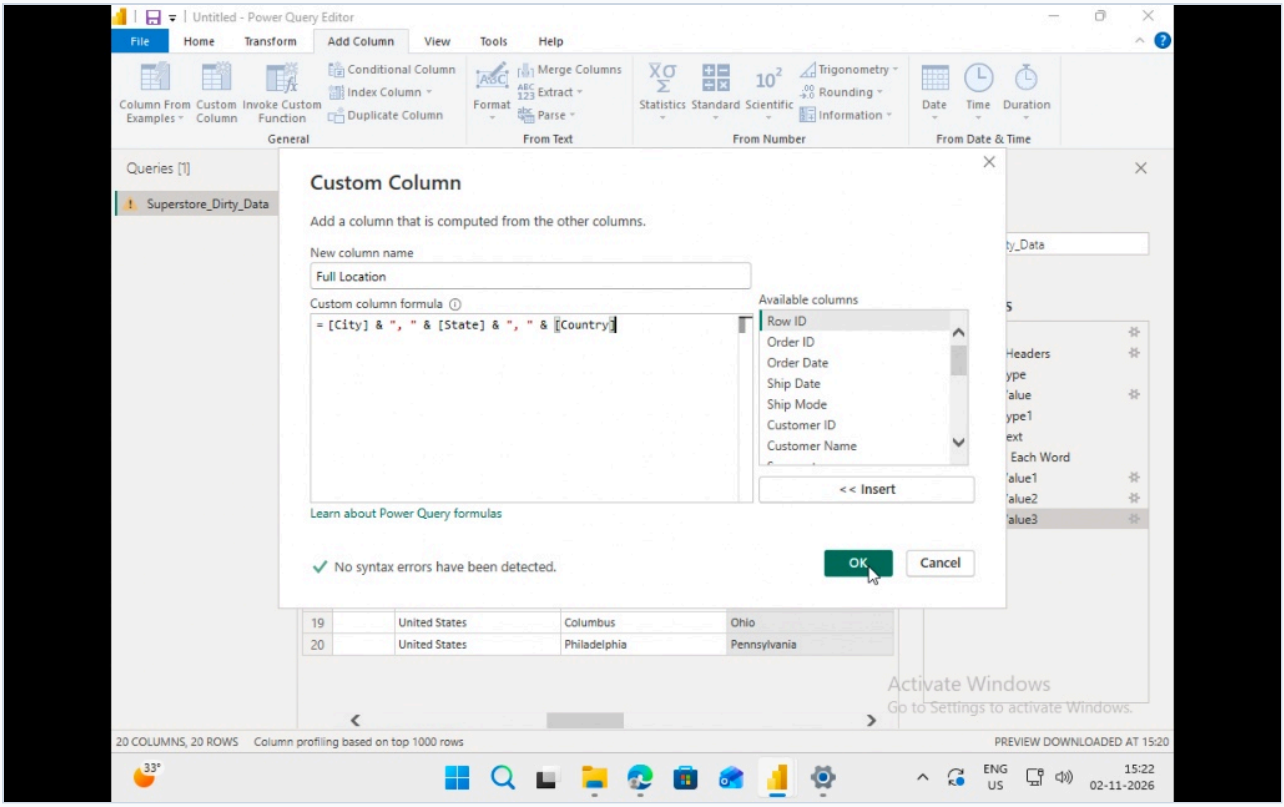


Figure 2.7: Creating the Full Location custom column in Power Query Editor (15:22).

Step 2.8: Add a Conditional Column for Profit Status

On the **Add Column** tab, click **Conditional Column**. Keep the column name as `Custom` and define the rule: If `Profit` *is greater than* `0`, Then output `Profitable`, Else output `Loss`. Click **OK**.

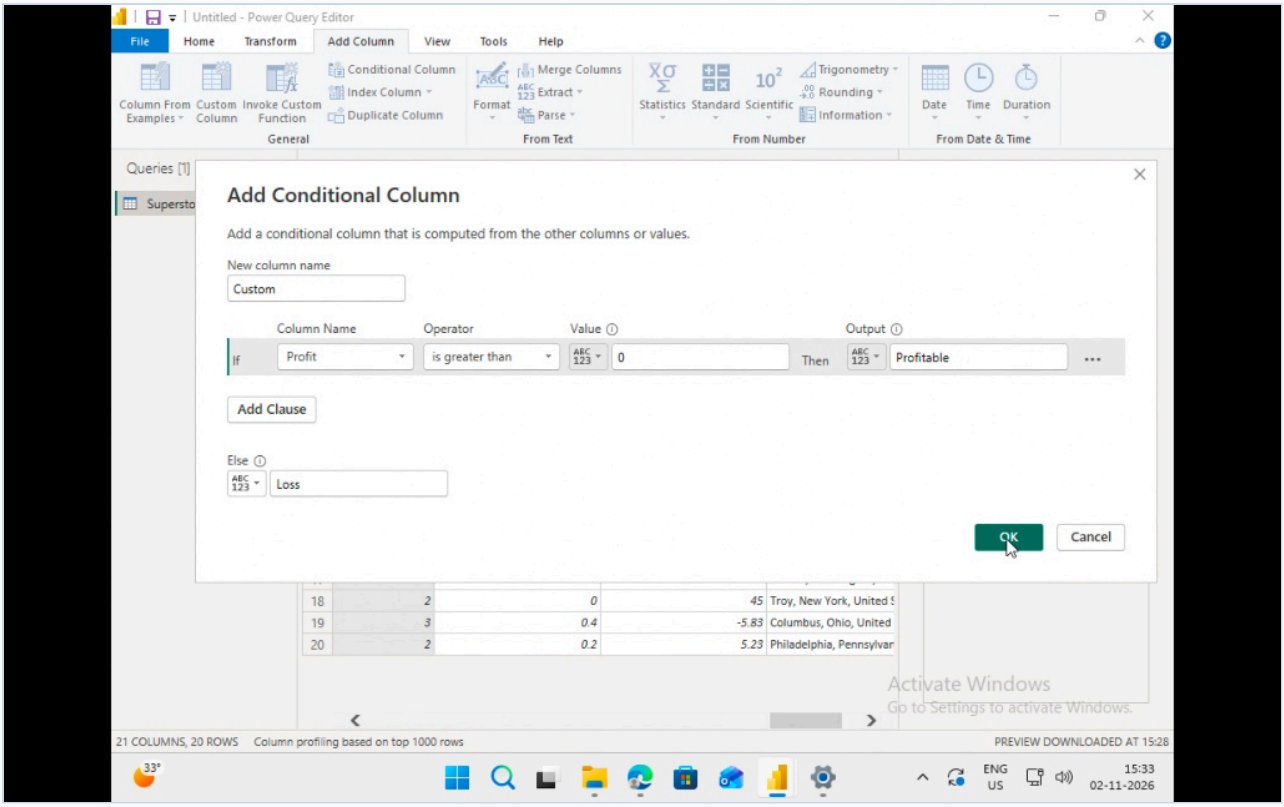


Figure 2.8: Configuring the Add Conditional Column dialog for Profit status (15:33).

Step 2.9: Verify Applied Steps and Close & Apply

Review the newly generated `Full Location` and `Custom` columns along with the complete sequence of transformations under **Applied Steps** in the Query Settings pane. Click **Home > Close & Apply** to load the cleaned dataset into the Power BI data model.

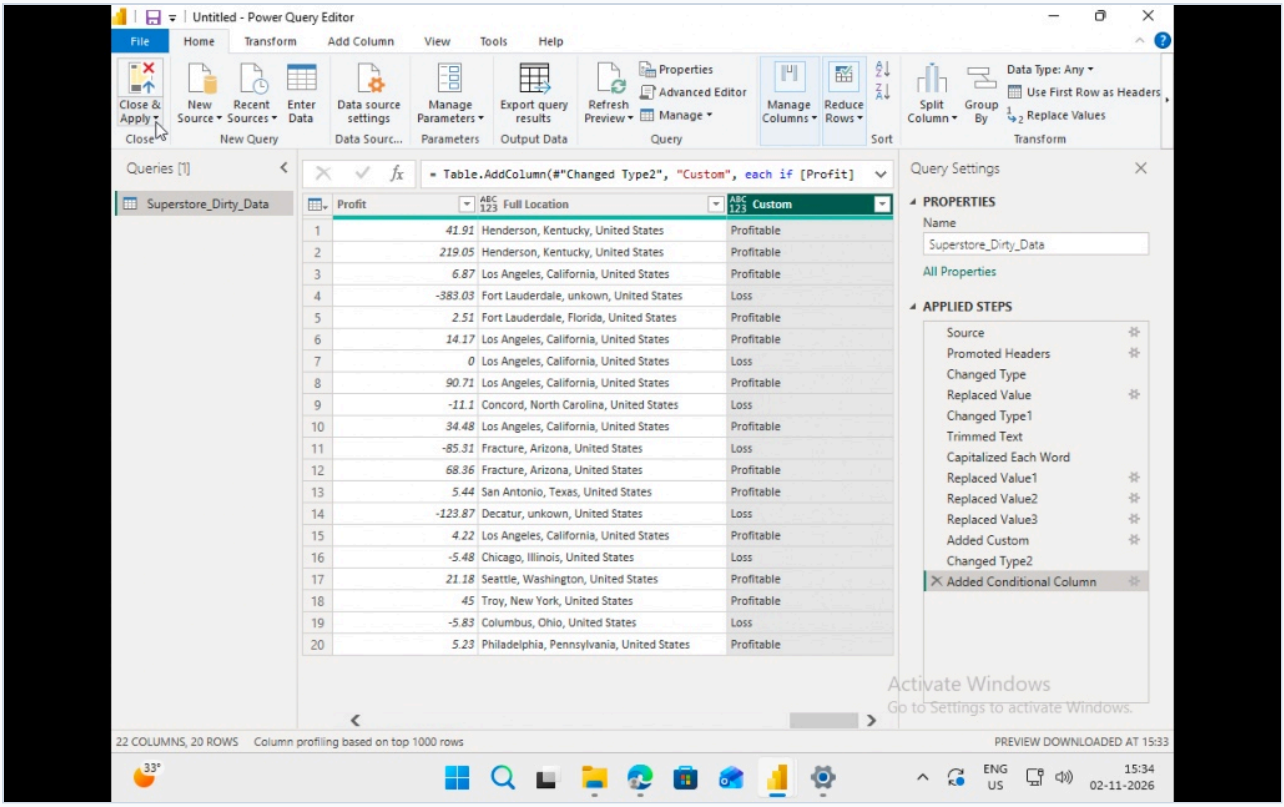


Figure 2.9: Reviewing all cleaned columns and clicking Close & Apply (15:34).

Task 3: Build a Data Model and Create Relationships

Step 3.1: Create a Dedicated Calendar Date Table Using DAX

Switch to the **Table view** in Power BI Desktop, go to the **Table tools** ribbon, and click **New table**. Enter the following DAX expression to generate the `DimDate` dimension table: `DimDate = ADDCOLUMNS(CALENDAR(MIN('Superstore_Dirty_Data'[Order Date]), MAX('Superstore_Dirty_Data'[Order Date])), "Year", YEAR([Date]), "Month Number", MONTH([Date]), "Month Name", FORMAT([Date], "mmmm"), "Month-Year", FORMAT([Date], "mmm yyyy"), "Quarter", "Q" & FORMAT([Date], "Q"))`.

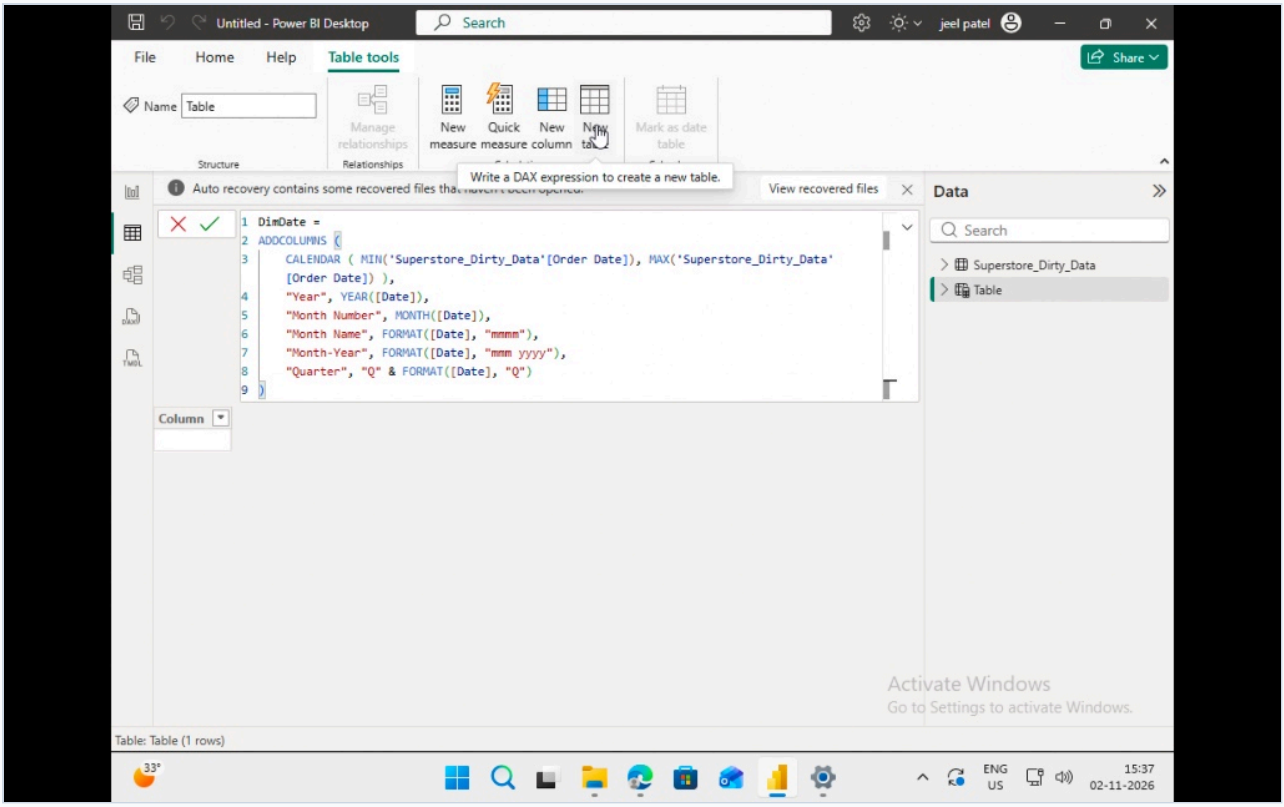


Figure 3.1: Writing the DAX expression to generate the DimDate calendar table (15:37).

Step 3.2: Sort Month Name Chronologically by Month Number

In the `DimDate` table, select the `Month Name` column. Under the **Column tools** tab, click **Sort by column** and select **Month Number** so months display in chronological order rather than alphabetical order.

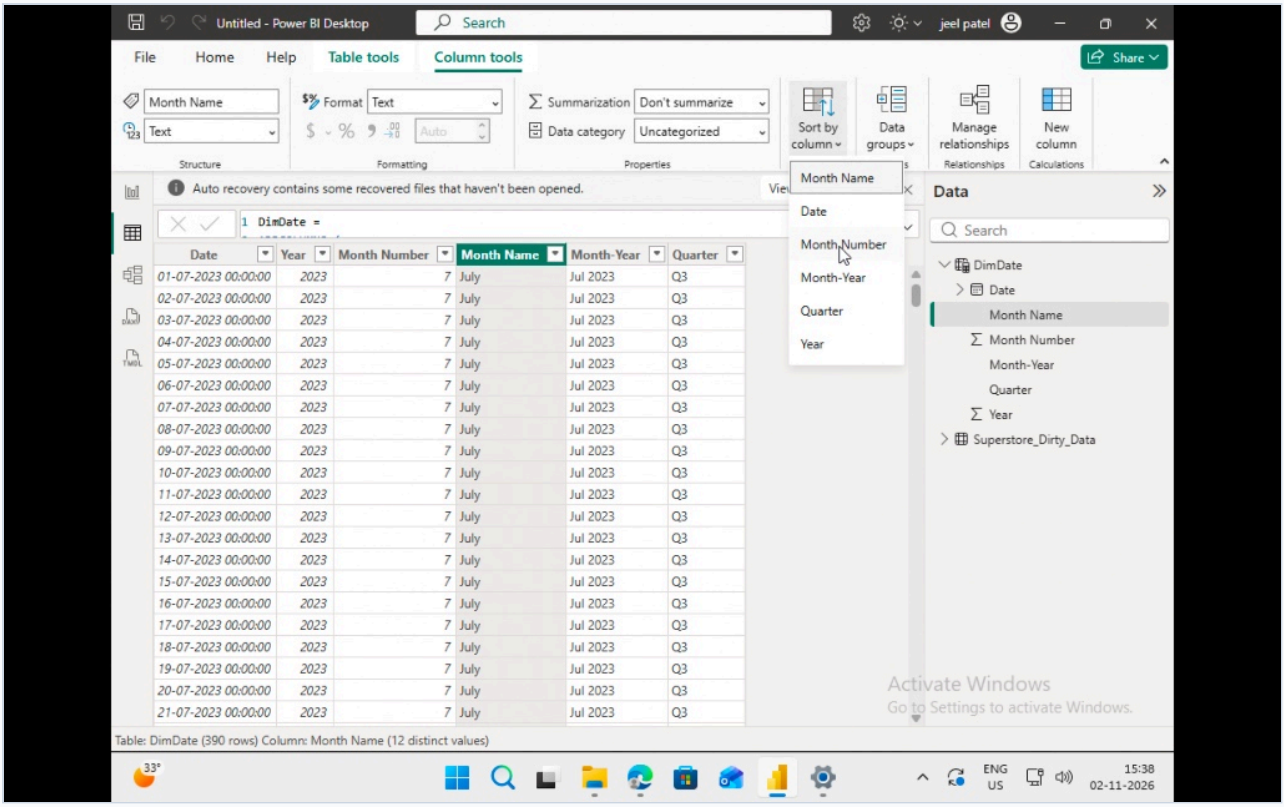


Figure 3.2: Sorting Month Name by Month Number in Column tools (15:38).

Step 3.3: Link DimDate to the Fact Table in Model View

Open the **Model view** from the left navigation bar. Drag the `Date` column from the `DimDate` table and drop it directly onto the `Order Date` column in the `Superstore_Dirty_Data` table.

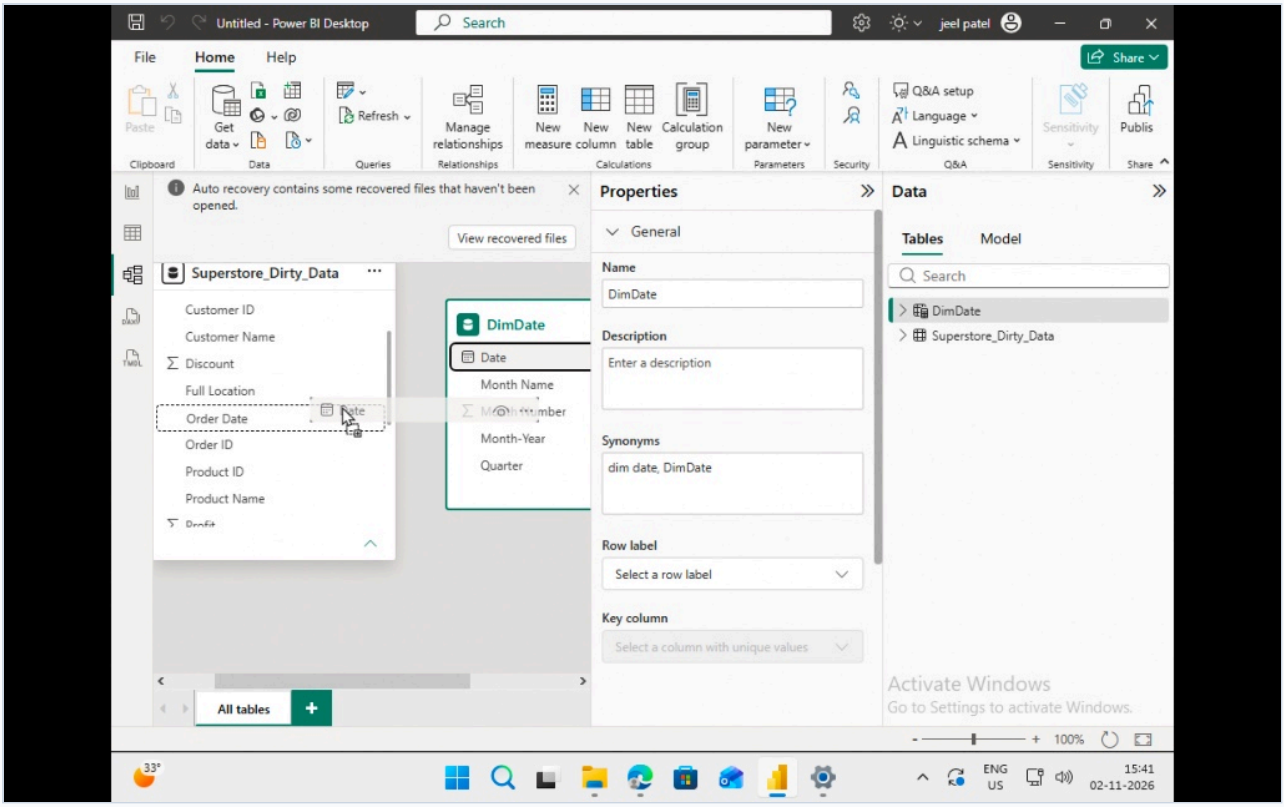


Figure 3.3: Creating a relationship between DimDate[Date] and Superstore_Dirty_Data[Order Date] (15:41).

Step 3.4: Configure Relationship Cardinality and Save

In the **New relationship** dialog window, verify that `DimDate[Date]` links to `Superstore_Dirty_Data[Order Date]` with a **Cardinality** of `One to many (1:*)`, **Cross-filter direction** set to `Single`, and **Make this relationship active** checked. Click **Save**.

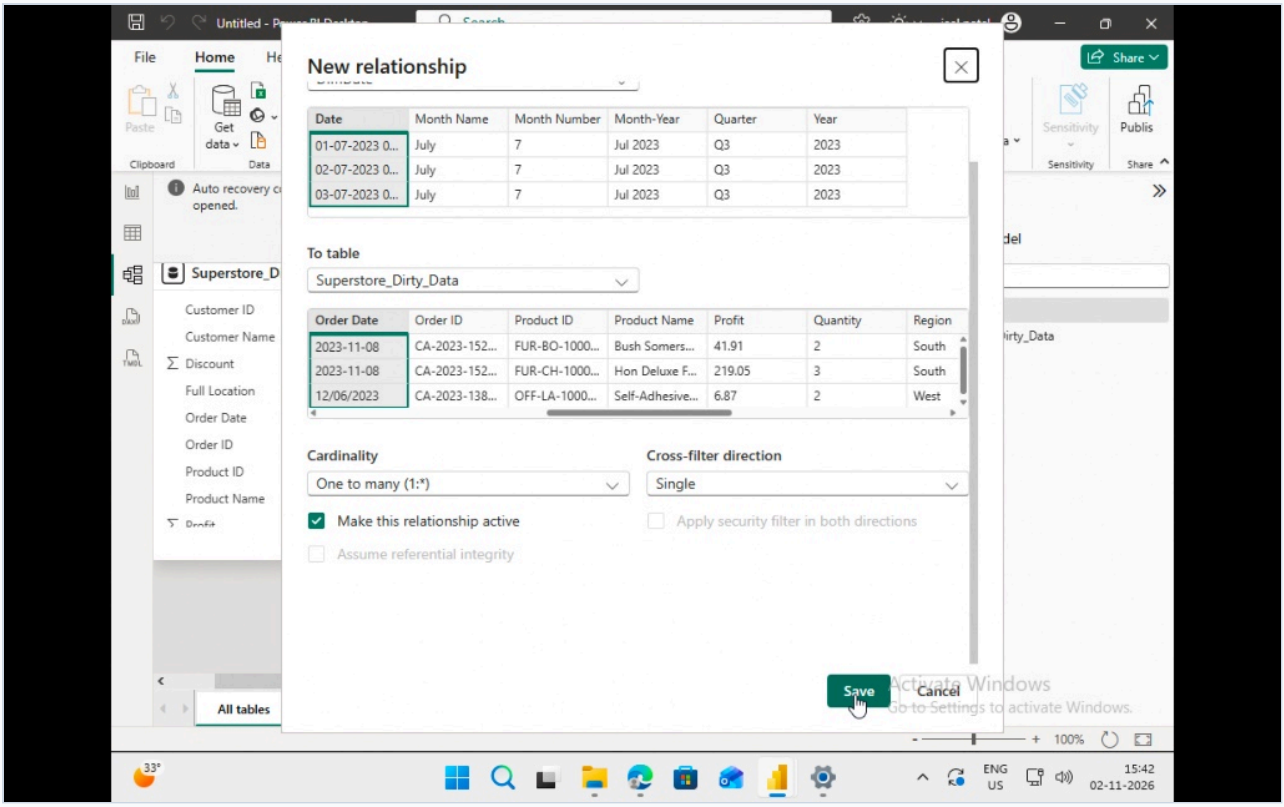


Figure 3.4: Saving the One-to-many (1:*) active relationship in the data model (15:42).

Task 4: Create Measures and Calculated Columns Using DAX

Step 4.1: Create Key DAX Measures for Sales, Profit, Orders, and AOV

After creating base measures for `Total Sales`, `Total Profit`, `Total Orders`, `Total Quantity`, and `Profit Margin %`, click **New measure** under **Measure tools** and enter the formula for Average Order Value: `Average Order Value = DIVIDE([Total Sales], [Total Orders], 0)`. Click the green checkmark to commit.

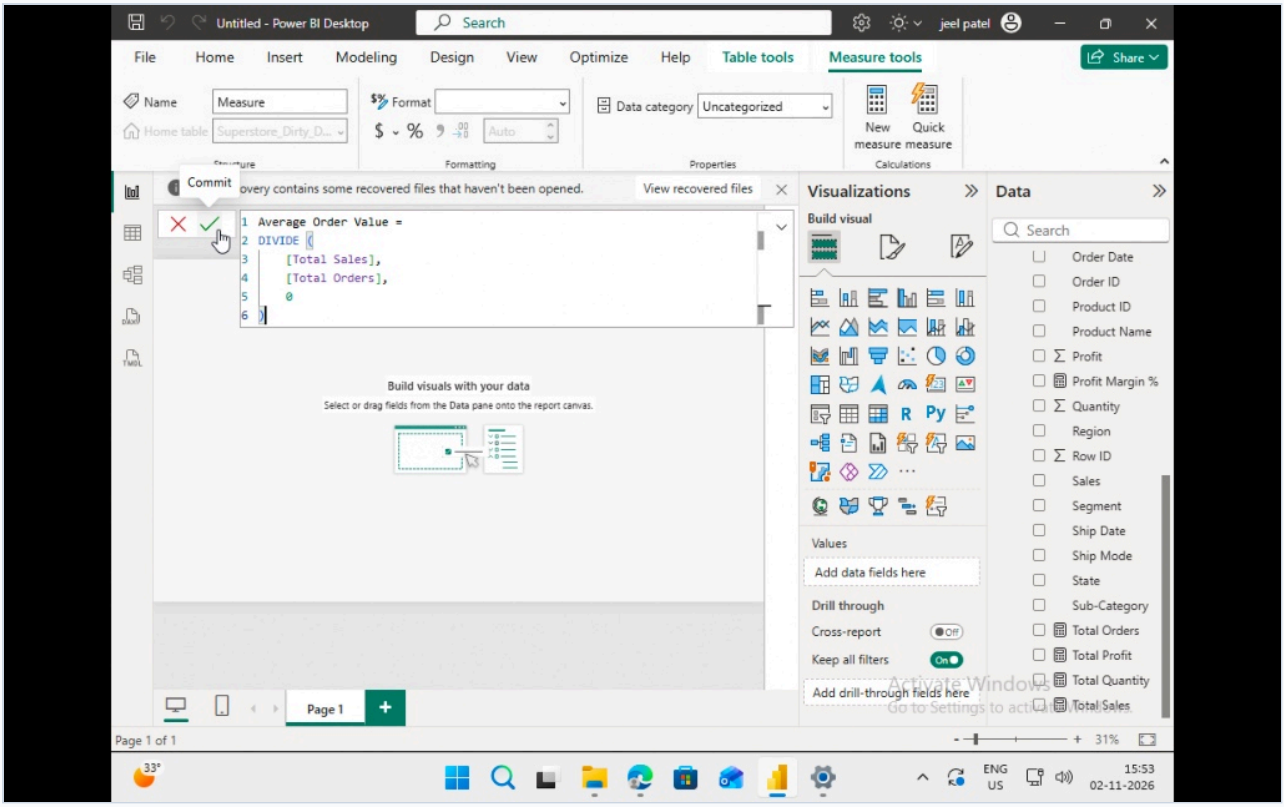


Figure 4.1: Defining and committing the Average Order Value DAX measure (15:53).

Step 4.2: Create a Calculated Column for Unit Price

Under **Table tools / Column tools**, click **New column** to add a row-level calculated column to `Superstore_Dirty_Data`. Enter the DAX formula: `Unit Price = DIVIDE('Superstore_Dirty_Data'[Sales], 'Superstore_Dirty_Data'[Quantity], 0)` and click Commit.

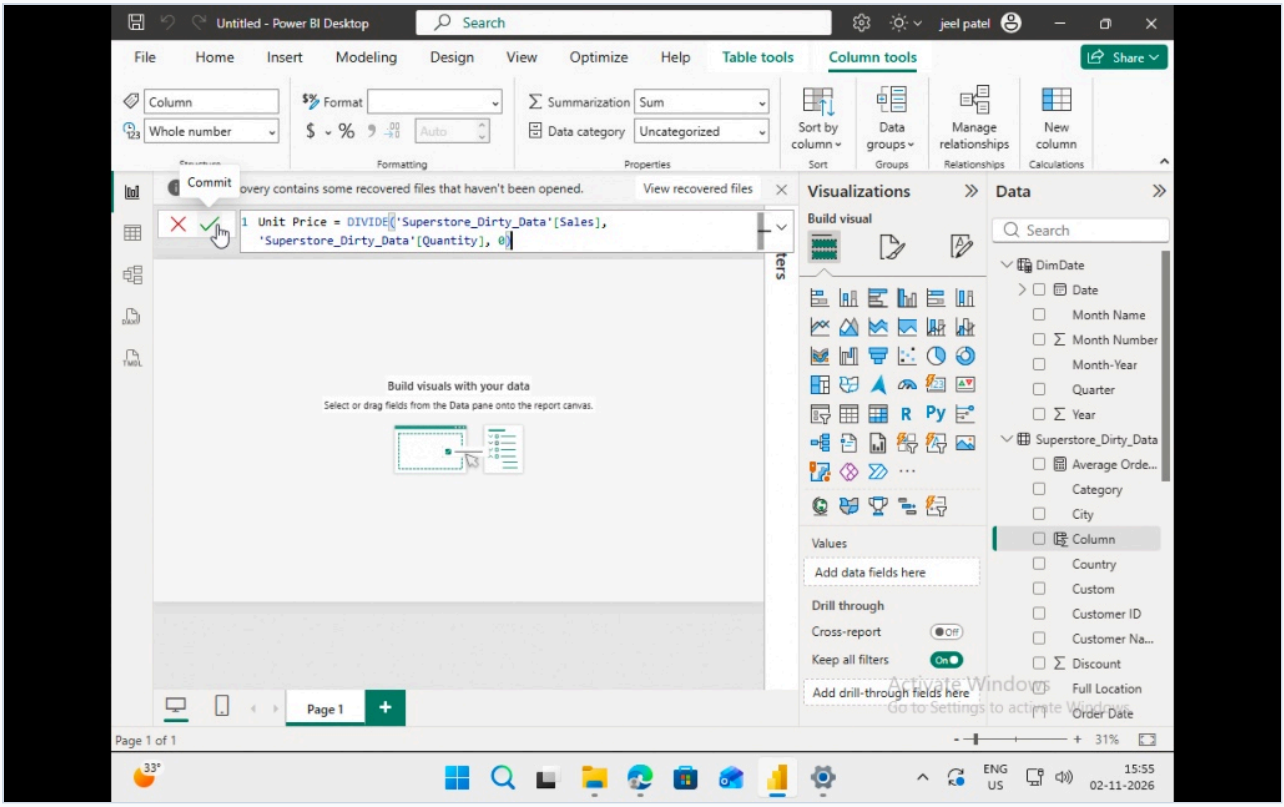


Figure 4.2: Creating the Unit Price calculated column using DAX DIVIDE (15:55).

Task 5 & Task 6: Design Interactive Dashboards, Filters & Slicers

Step 5.1: Build the Sales Performance Dashboard Layout

In the **Report view**, design the dark-themed **Sales performance Dashboard** featuring KPI Cards for `Total Orders` (16) and `Total Profit`, a Donut Chart showing `Total Orders by Product Name`, and a Clustered Column Chart displaying `Total Profit by Month-Year and Sales`, alongside interactive filtering and optimization controls under the **Optimize** ribbon.

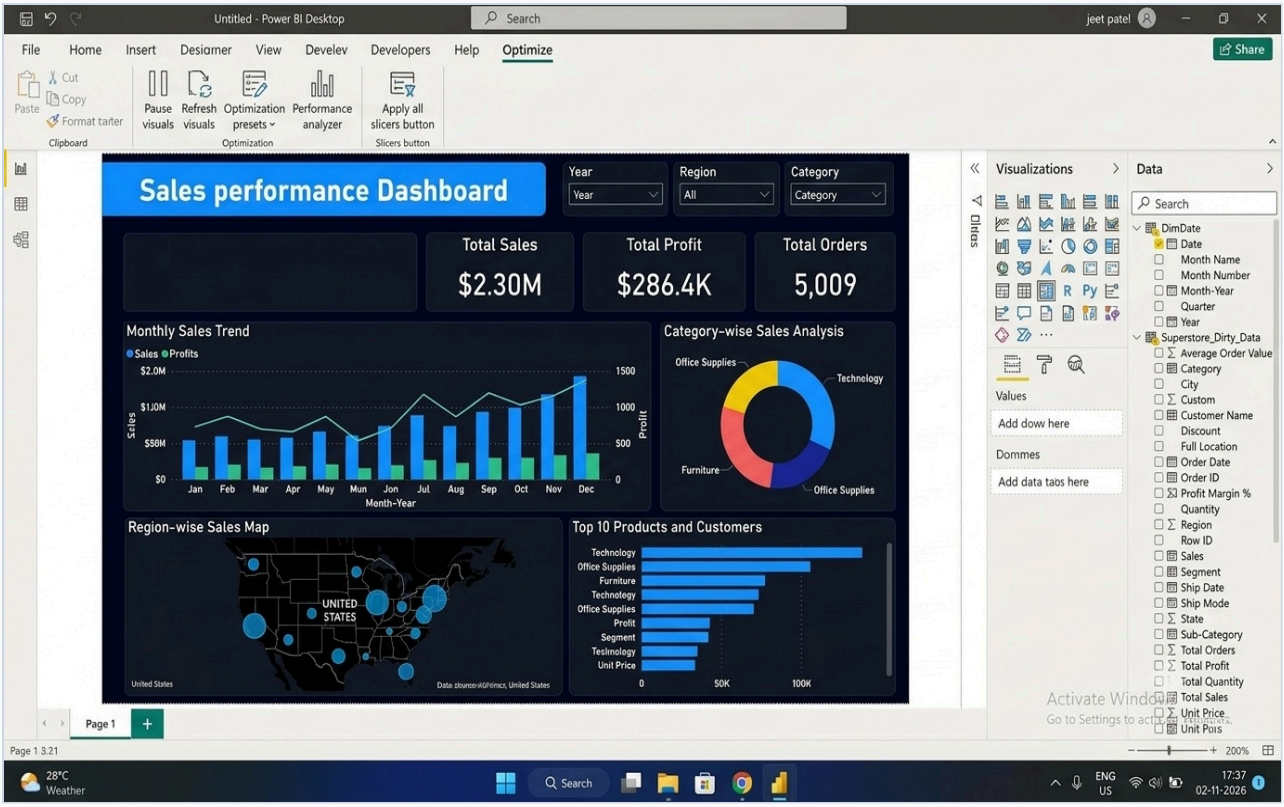


Figure 5.1: Completed interactive Sales Performance Dashboard in Power BI Desktop (17:37).

Task 7: Save Locally and Publish the Report to Power BI Service

Step 7.1: Save the Power BI Project (.pbix) File

Go to **File > Save as**, navigate to `Documents > PowerBI`, enter the file name `Sales_Performance_Dashboard` as a Power BI file (`*.pbix`), and click **Save**.

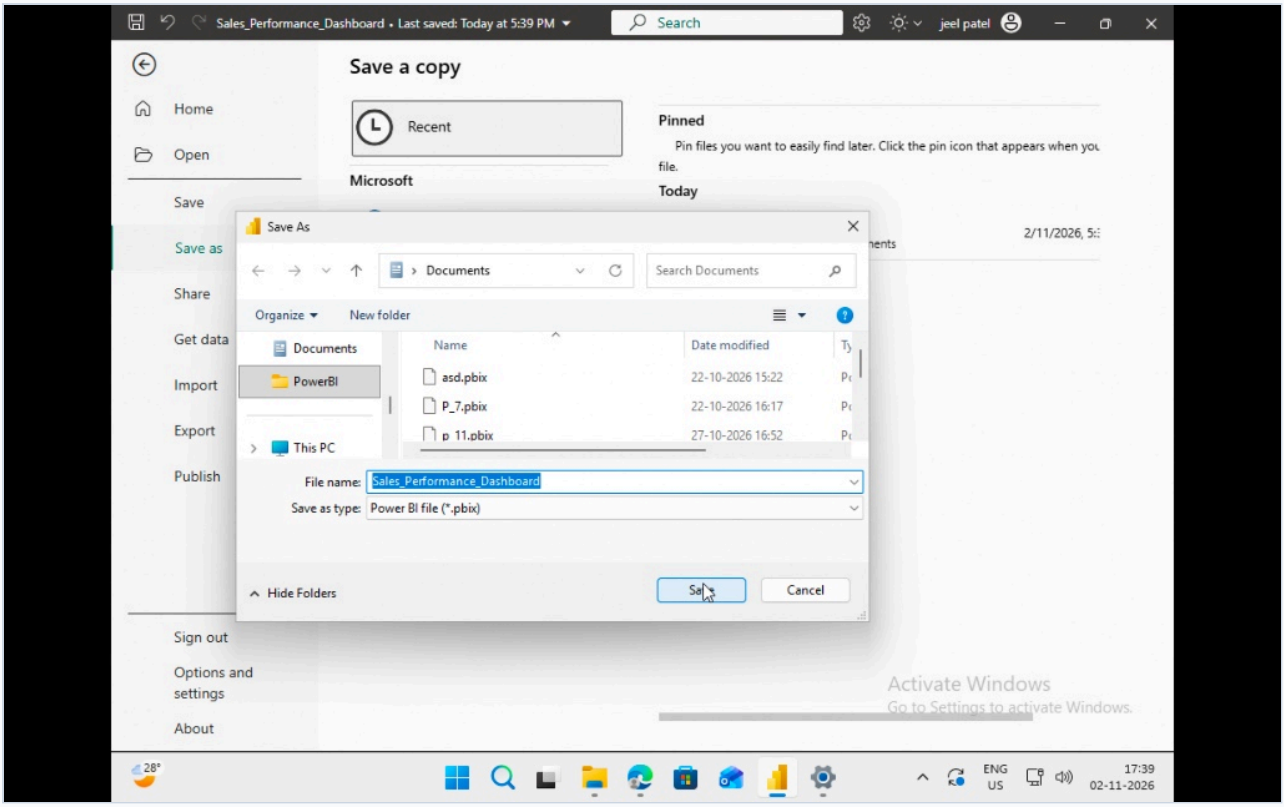


Figure 7.1: Saving the project locally as Sales_Performance_Dashboard.pbix (17:39).

Step 7.2: Select Destination Workspace to Publish

On the **Home** tab, click **Publish**. In the **Publish to Power BI** dialog box, select **My workspace** as the destination and click **Select**.

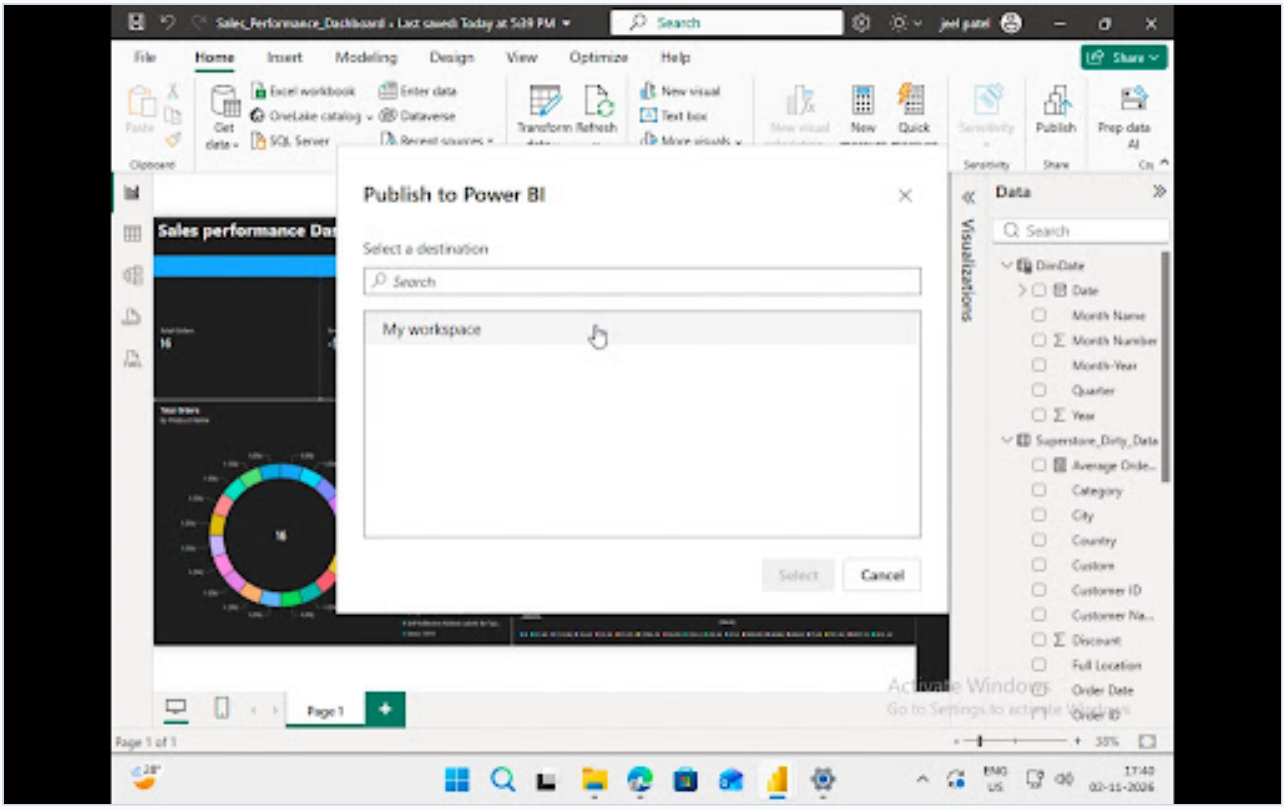


Figure 7.2: Choosing My workspace in the Publish to Power BI prompt (17:40).

Step 7.3: Upload Report to Power BI Cloud Service

Wait while Power BI Desktop packages and uploads `Sales_Performance_Dashboard.pbix` to the Power BI Service workspace.

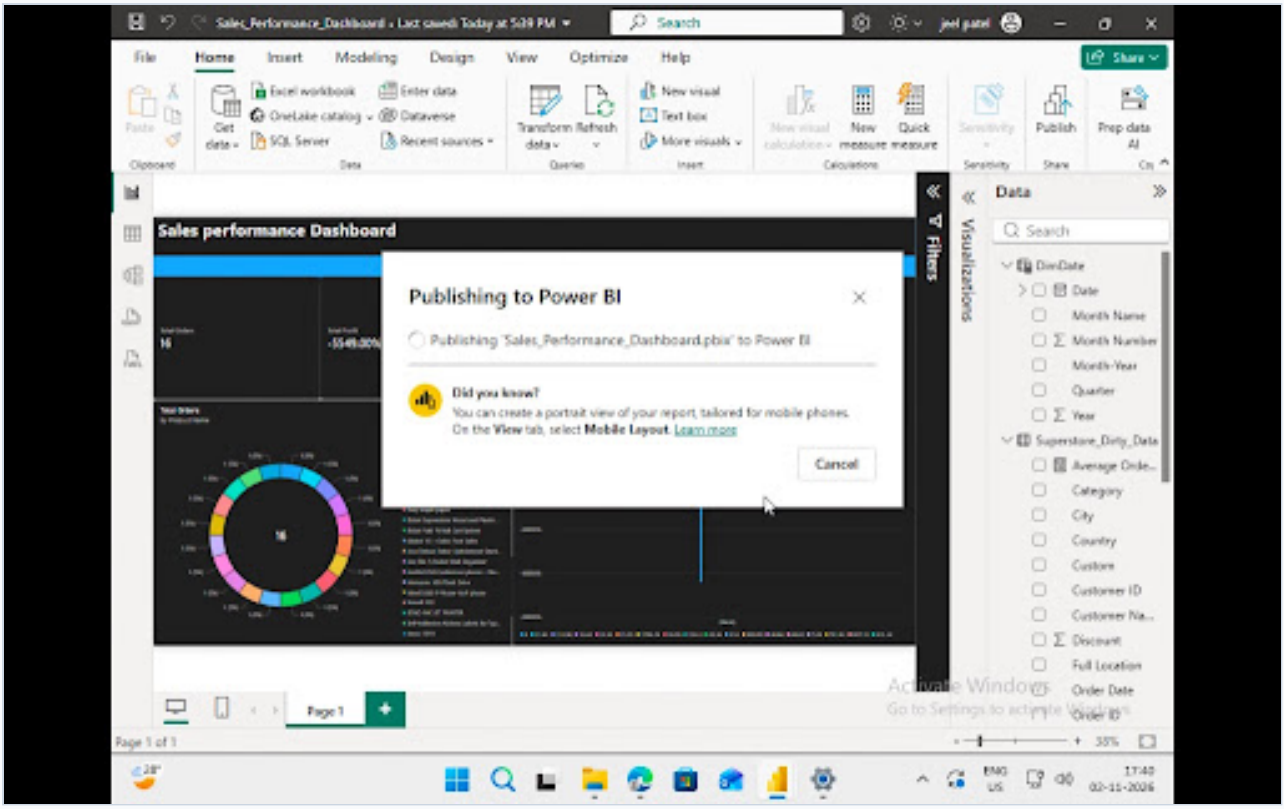


Figure 7.3: Publishing Sales_Performance_Dashboard.pbix to Power BI Service (17:40).

Task 8: Configure Data Refresh and Share the Dashboard

Step 8.1: Share the Published Dashboard via Power BI Service

Open the published report inside the **Power BI Service** browser portal (`app.powerbi.com`). Click the **Share** button in the top-right action bar, select the recipient (`jeel patel`) with viewing and sharing permissions, and click **Send** (or copy the link).

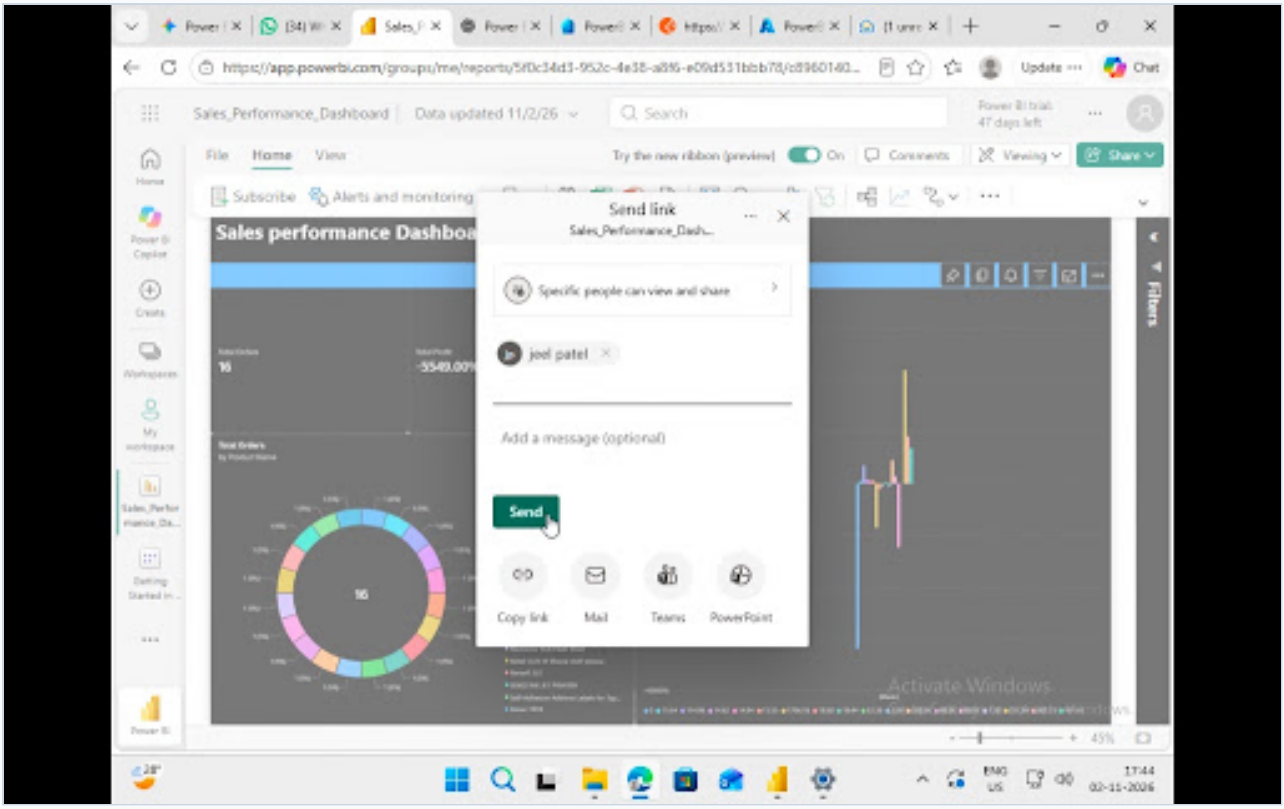


Figure 8.1: Sharing the Sales Performance Dashboard link in Power BI Service (17:44).

Step 8.2: Download the On-Premises Data Gateway

To enable automated cloud refresh for the local CSV file, click the **More options (...)** menu in the top-right corner of Power BI Service, hover over **Download**, and click **Data Gateway**.

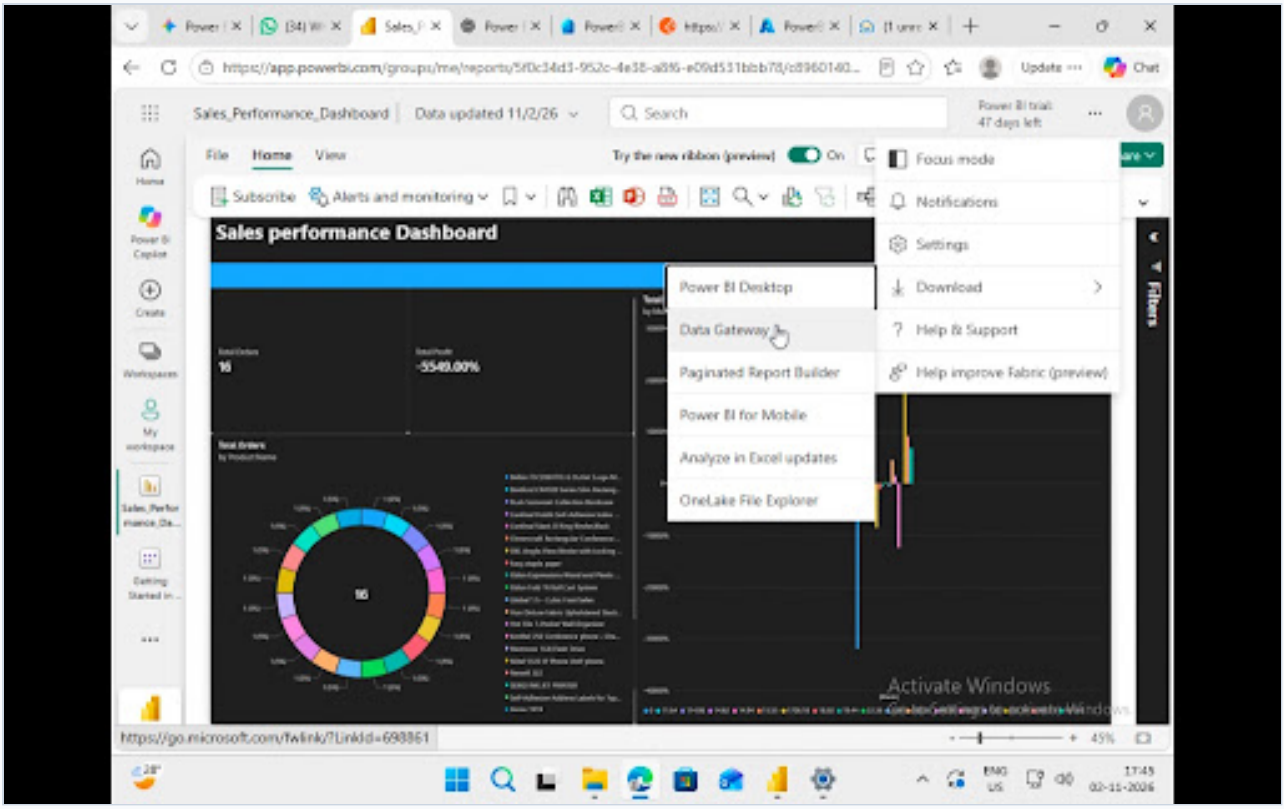


Figure 8.2: Selecting Download > Data Gateway from the Power BI Service menu (17:45).

Step 8.3: Launch Gateway Registration on the Local PC

Run the On-premises data gateway installer and sign in as `jeel070918@zohomail.in`. Select the option **Register a new gateway on this computer** and click **Next**.

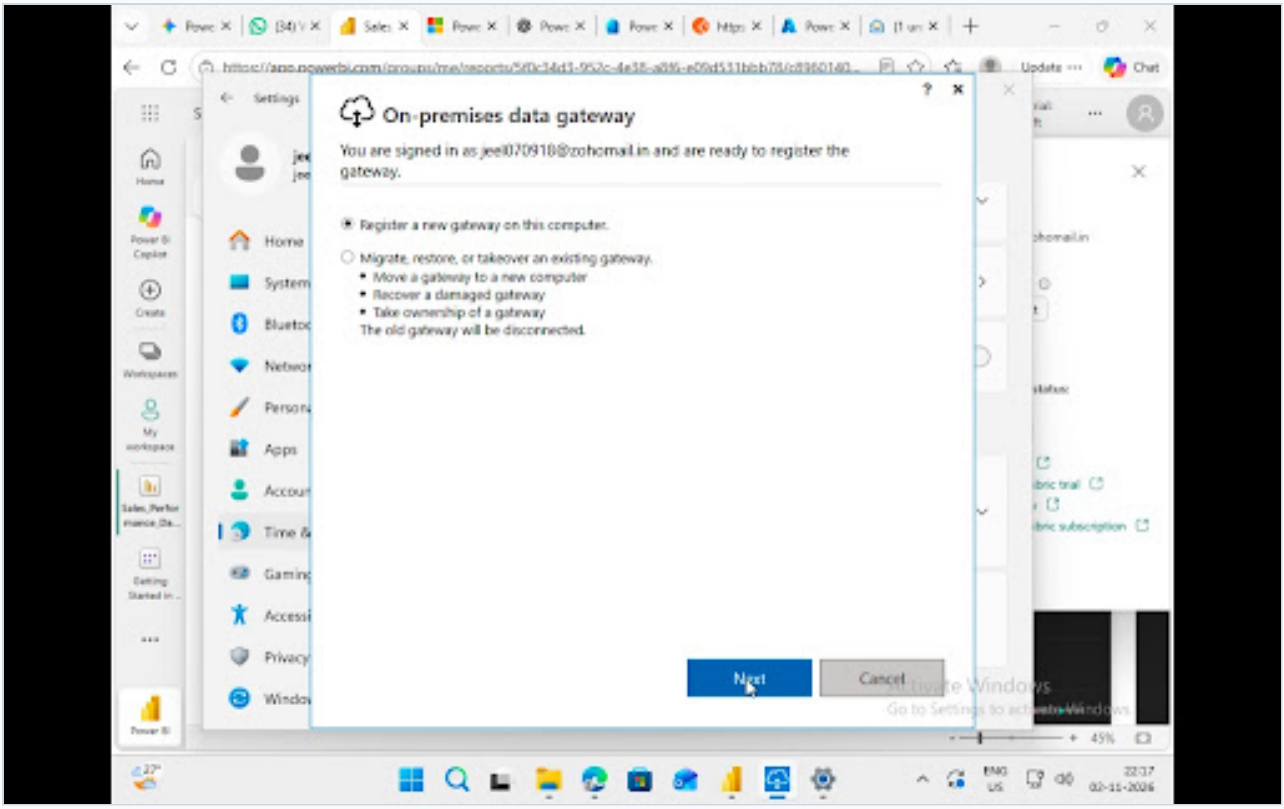


Figure 8.3: Selecting Register a new gateway on this computer (22:17).

Step 8.4: Configure Gateway Name and Recovery Key

Enter `jeel patel` as the **New on-premises data gateway name**, create and confirm the 8-character minimum **Recovery key**, verify the cloud region (Central India), and click **Configure**.

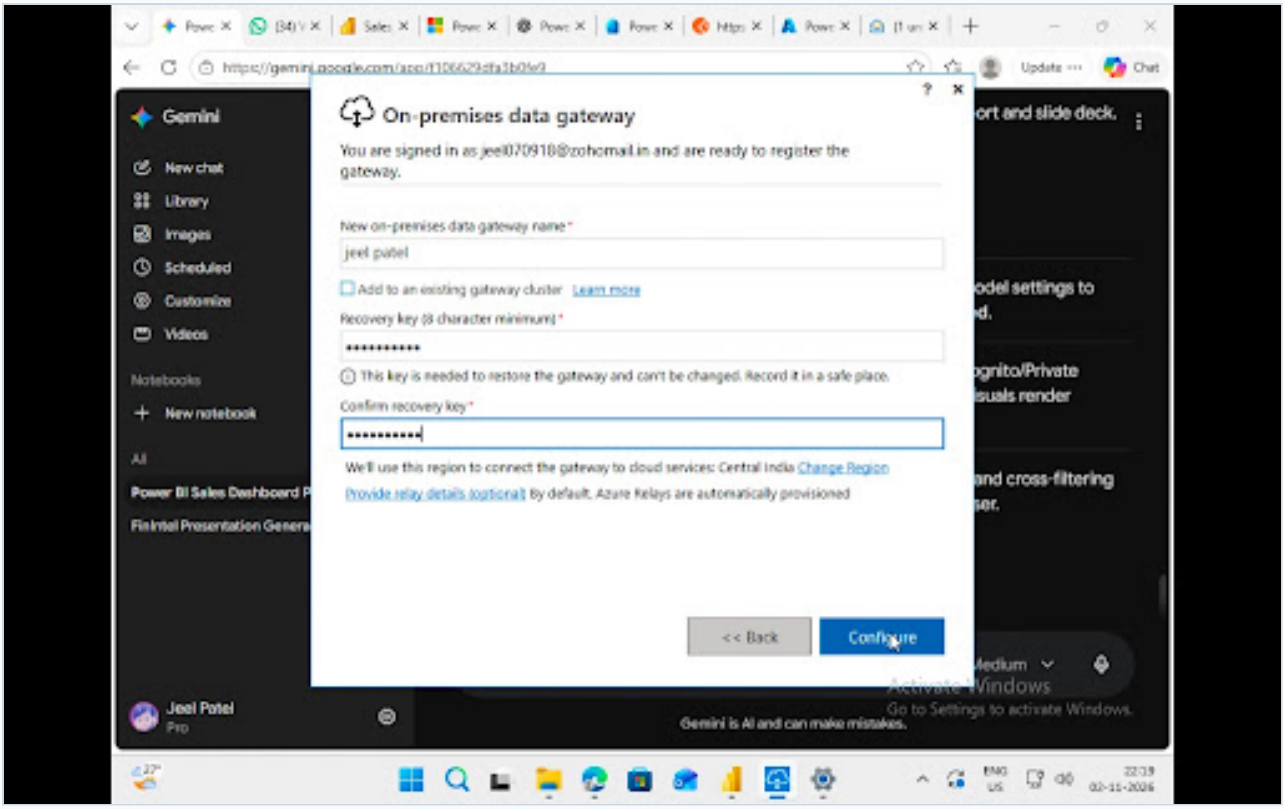


Figure 8.4: Entering the gateway name and recovery key and clicking Configure (22:19).

Step 8.5: Verify Gateway Online Status and Map Semantic Model

Confirm that the gateway `jeel patel` displays the green checkmark status: *"The gateway 'jeel patel' is online and ready to be used."* In the background Power BI Service **Settings / Sales Performance Dashboard** page, expand **Gateway and cloud connections** and map the dataset to the online gateway `jeel patel`.

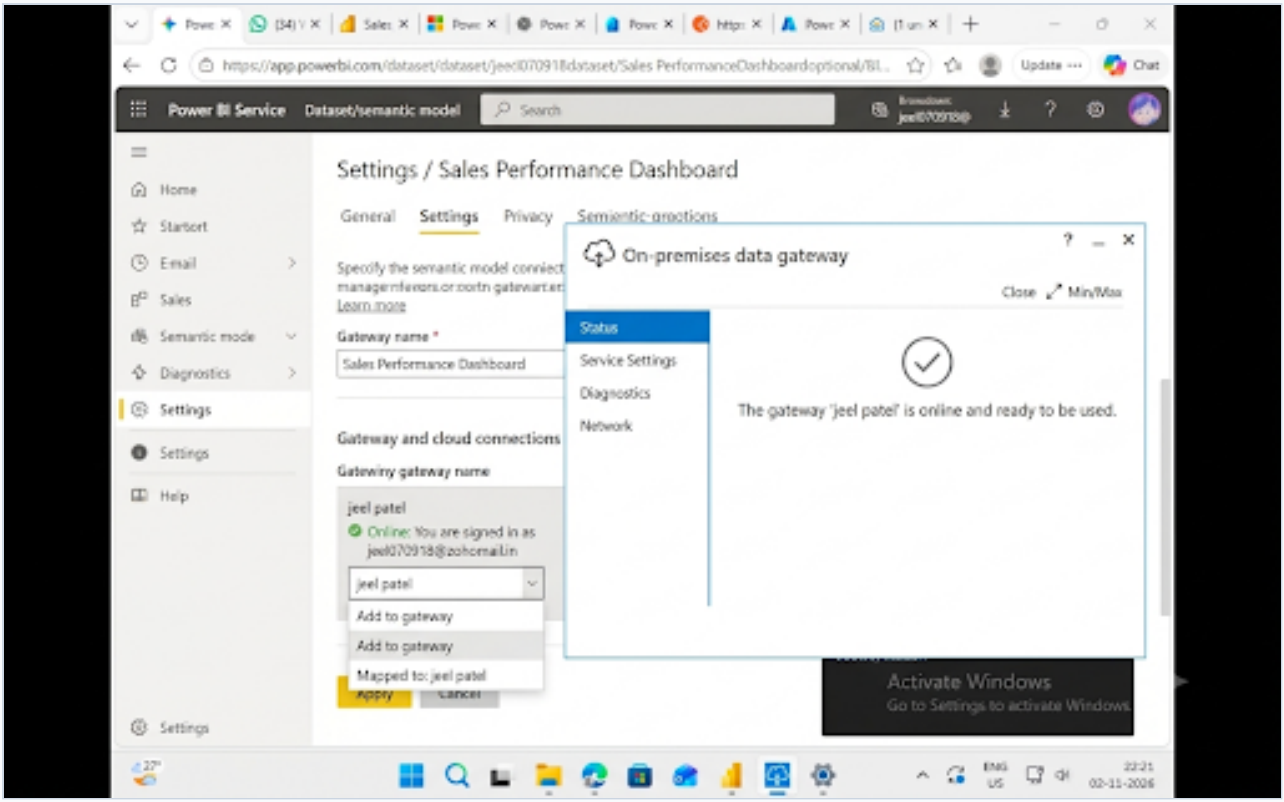


Figure 8.5: Verifying the gateway is online and mapping the semantic model connection (22:21).

Step 8.6: Configure Daily Scheduled Refresh

In the semantic model **Settings for Sales Performance Dashboard** page, expand the **Scheduled refresh** section. Toggle **Keep your data up to date** to **On**, set **Refresh frequency** to `Daily`, select the time zone `(UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi`, schedule the time for `08:00 AM`, check failure notifications, and click **Apply**.

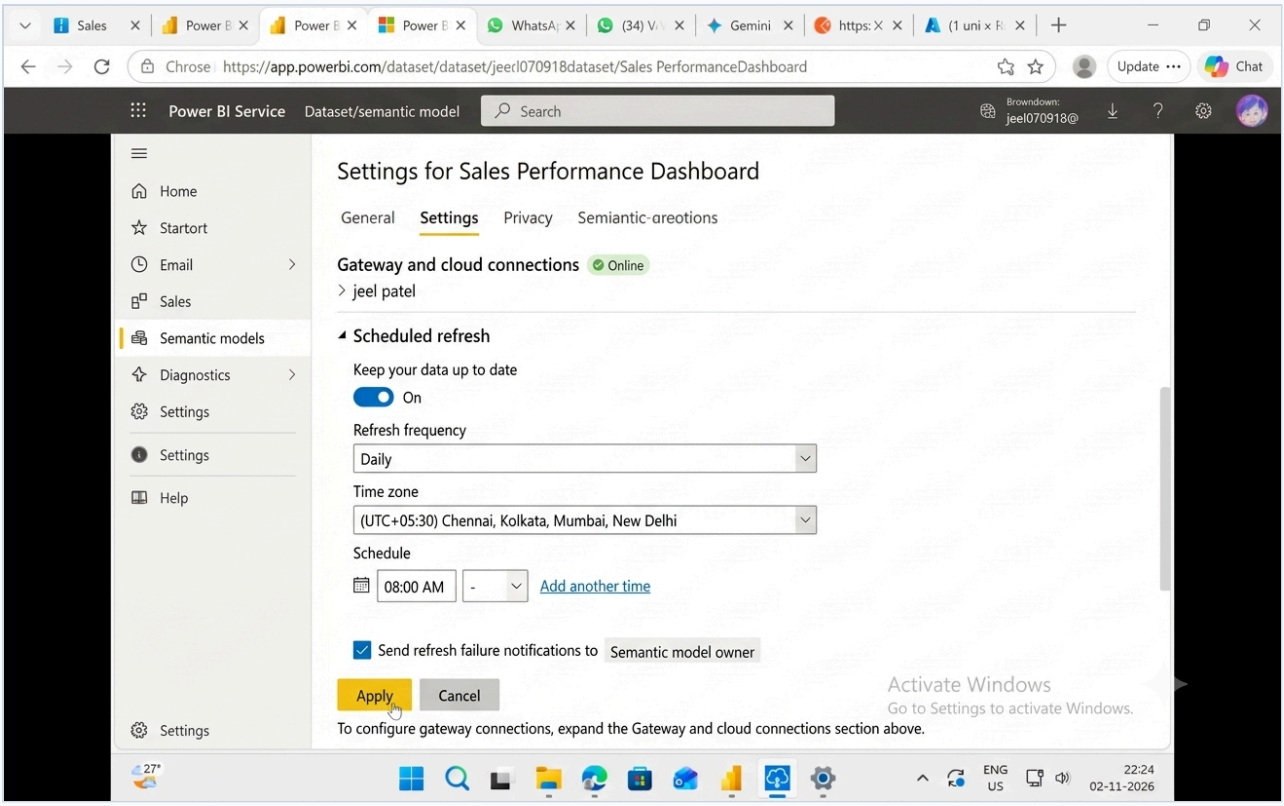


Figure 8.6: Enabling Daily Scheduled Refresh at 08:00 AM IST and clicking Apply (22:24).

Step 8.7: Confirm Completed Status in Refresh History

Open the **Refresh history** dialog for the `Sales Performance Dashboard` semantic model to verify that the scheduled/on-demand refresh executed from `11/2/2026, 10:24:00 PM` to `11/2/2026, 10:25:01 PM` shows a green checkmark with status **Completed**.

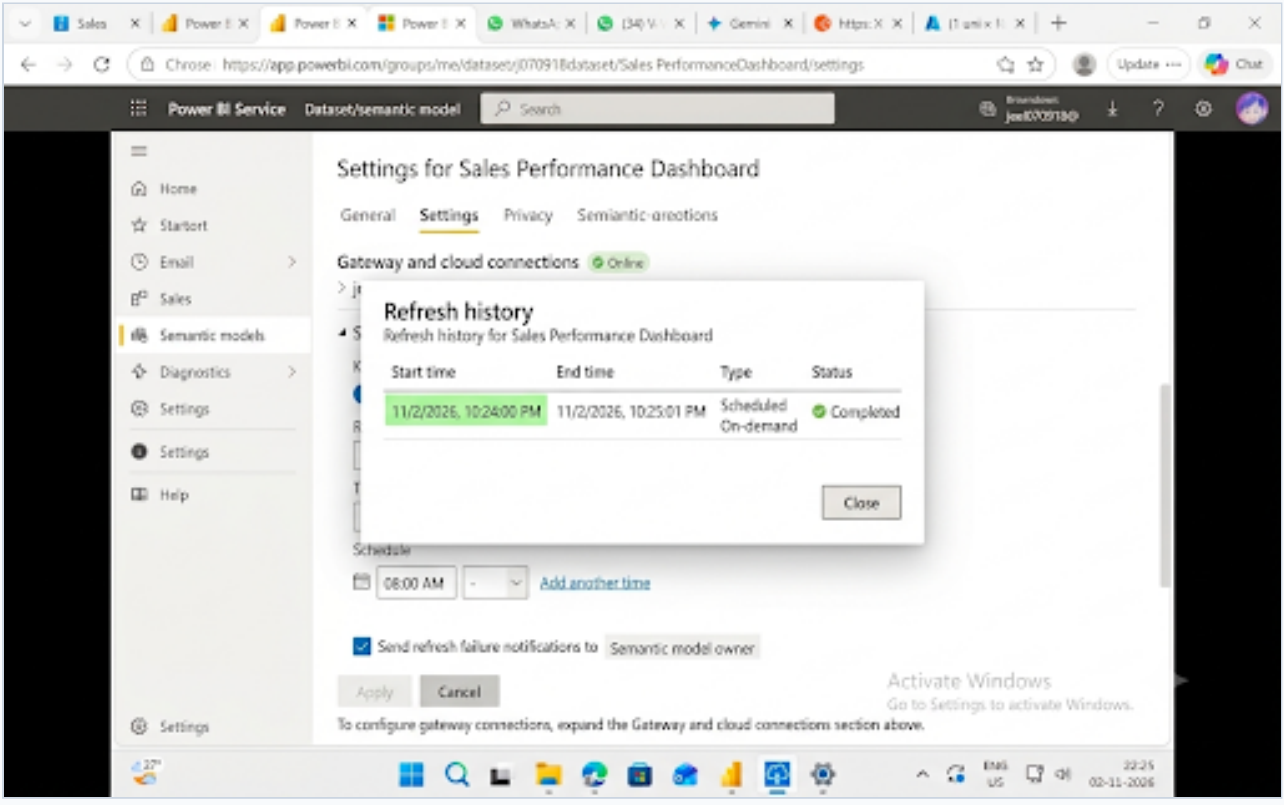


Figure 8.7: Verifying the Completed data refresh status in Refresh history (22:25).